FilmDB 数据库性能实验

文件访问与关系数据库的性能及一致性比较

姓名: 学号:

01 主要结论

对于数据可驻留内存、查询已经预热的读多写少负载，在相同 CPU 配额和所测数据库配置下，PostgreSQL 相比 openGauss 的主键点查延迟更低，并发只读及 90% 读、10% 单行同步更新的吞吐量更高。本实验据此支持在这些条件下优先选择 PostgreSQL。串行测试中，PG 的全表搜索、聚合和批量更新耗时也低于 OG；固定返回 54 行的索引连接则由 OG 领先。

文件与数据库的选择取决于访问方式及一致性要求。单进程访问静态数据、仅按编号读取单条记录、偏移索引已装载时，文件索引的查询耗时最低；需要维护多表引用和撤销失败事务时，数据库提供了本文件程序不具备的约束与回滚能力。

比较条件	主要结果
静态记录，按编号单条读取	文件索引 4.84 µs；PG 97.43 µs；OG 194.43 µs。
固定返回 54 行的关联查询	文件扫描 1.81 s；索引连接 PG 1.42 ms、OG 1.07 ms。
批量字符串替换，同步完成更新	PG 1.47 s；文件 1.83 s；OG 3.16 s。
外键校验及事务失败后回滚	两库均拒绝非法引用，回滚后数据恢复原值。

PG 为 PostgreSQL 14.24，OG 为 openGauss 6.0.1。表中性能数据来自 100 倍规模，点查为每轮平均单次耗时的三轮中位数，其余为操作耗时的三轮中位数。查询不含索引建立、装载和连接准备。

数据库如何处理一次失败的跨表操作

事务先将一部电影的片长从原值改为新值，再插入引用不存在电影的演职关系。两库均拒绝该插入；执行回滚后，先前成功的片长修改也被撤销。

检查步骤	PostgreSQL 与 openGauss 的实际结果
事务内读取片长	可见修改后的值，确认第一步已执行。
插入非法演职关系	外键错误 23503，拒绝不存在的电影编号。
回滚并重新读取	片长恢复，四张表的内容摘要均与操作前一致。

数据库统一执行引用约束和事务撤销。文件程序若要提供同样的跨表保证，需要另外实现检查与恢复逻辑；本程序的单文件临时写入与替换没有实现这些功能。

结论范围

上述实验覆盖主键读取、全表子串搜索、低结果量连接、分组聚合及更新，均使用可驻留内存的数据和预热后的访问。性能关系还受数据分布与资源配置影响；连接结果尤其包含索引与扫描的差异。实验没有覆盖超内存数据、长期写入或崩溃恢复，也未证明两库在这些场景中的性能关系。

02 访问路径对查询性能的影响

同一份文件，顺序点查约需 263 ms，加入偏移索引后只需 4.84 µs。 顺序扫描与索引定位产生了显著的性能差异。关联查询同样比较了文件扫描与数据库索引访问，因此结果反映的是这些具体实现的差异。

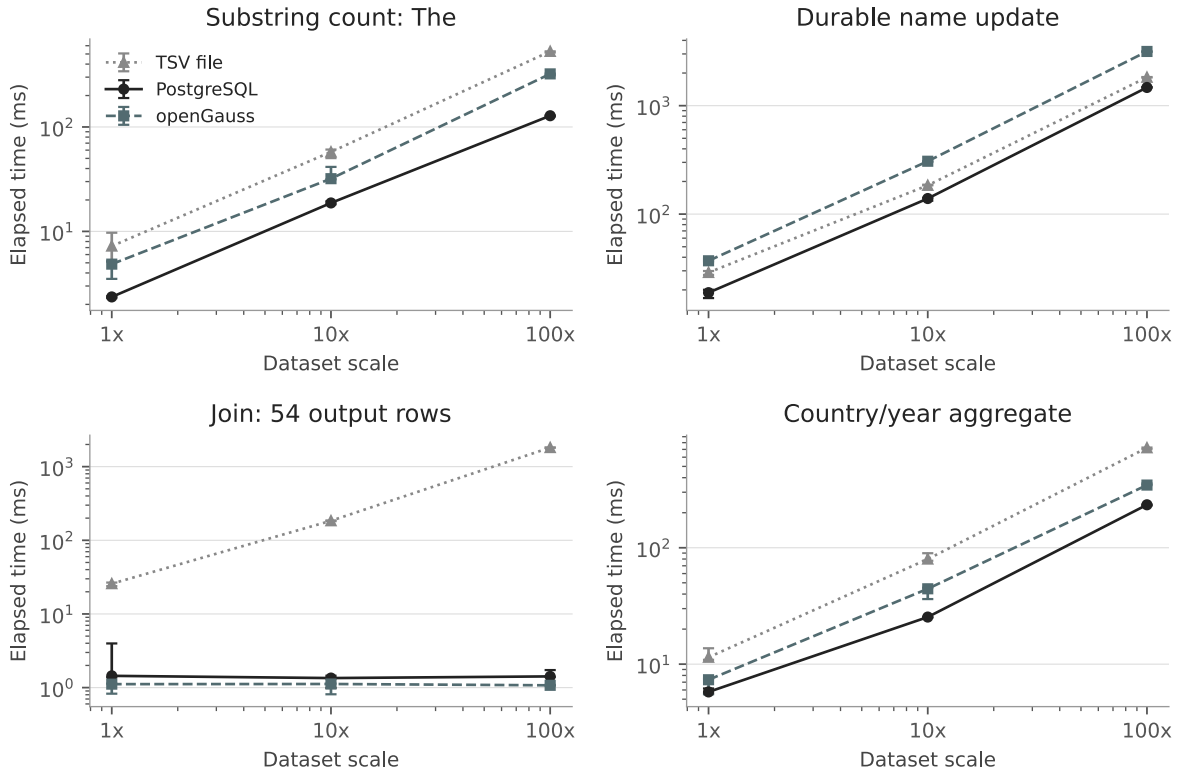


图 1　从 1 倍扩到 100 倍数据，扫描和更新耗时增长；固定返回 54 行的索引连接变化较小。横纵轴均为对数刻度，误差线为三轮最小值至最大值。

人物连接中，文件程序依次扫描演职关系表和电影表；数据库先通过人物索引找到关联记录，再用电影主键读取详情。100 倍数据下，文件扫描耗时约 1.81 秒，数据库索引连接约 1 毫秒。由于三个规模都返回 54 行，这项测试考察的是固定结果量下的查询性能，而非连接结果随数据规模增长的情况。

标题计数和分组聚合仍需扫描电影表。文件程序逐行解析 TSV，聚合用 `std::map` 累计；两库采用 HashAggregate。100 倍数据下，PG 聚合用时 234 ms，约为文件程序的三分之一。这里同时存在数据格式和算法实现差异，不能把收益全部归因于数据库存储。

已装载索引下的单条记录读取

100×，按编号查询	平均单次耗时 / µs	三轮范围 / µs
文件扫描	262765.74	[262640.25, 264006.09]
文件偏移索引	4.84	[4.67, 4.95]
PostgreSQL	97.43	[92.14, 98.34]
openGauss	194.43	[137.52, 195.71]

每轮 100 次，三轮使用相同种子；耗时列取每轮平均值的中位数，范围为［最小值，最大值］，不含请求准备、连接及索引装载或建立。文件索引整轮仅约 0.48 ms，精确数值的代表性仍需更长批次和不同请求序列验证。

文件索引把编号映射到文件偏移，查内存哈希表后直接读取一条记录；数据库还涉及客户端与服务端通信及语句执行。文件索引的建立另需 0.946 秒，因此 4.84 µs 适用于已经准备好的索引。标题搜索和人物连接没有使用这份索引，它们需要不同的访问路径。

03 扫描、更新与混合负载比较

在预热后的全表子串搜索、分组聚合及批量字符串替换中，PG 的端到端耗时均低于 OG。 对于返回 54 行的索引连接，OG 耗时更低。下表保留文件实现作为参照，比较各类操作的性能差异。

100× 耗时 / ms	文件	PostgreSQL	openGauss
The 子串计数	529.23 [515.62, 529.85]	128.13 [126.06, 128.68]	322.35 [312.23, 324.21]
姓名同步更新	1827.72 [1819.72, 1836.73]	1472.21 [1471.99, 1481.63]	3162.42 [3132.07, 3169.66]
人物连接	1814.34 [1811.70, 1816.68]	1.42 [1.39, 1.73]	1.07 [1.06, 1.09]
国家 / 年份聚合	722.42 [703.17, 726.08]	234.02 [230.59, 234.67]	345.67 [345.26, 356.47]

三轮中位数，下方为［最小值，最大值］。

批量更新的端到端耗时

把姓名中的大写 T 替换成 Ttt，100 倍数据共修改 128,700 行。文件重写整张人物文件，同步后替换原文件；数据库更新匹配记录并提交事务。PG 用时 1.47 秒，OG 用时 3.16 秒，后者也慢于文件的 1.83 秒。数据库无需重写整张人物表，但仍包含记录定位、修改与事务提交等开销。实验未分解两库的内部写入开销，因此不能仅凭结果确定差距来源。

标题搜索与结果集大小

100× 关键词	返回行数	文件 / ms	PG / ms	OG / ms
The	156,300	554.47	209.60	402.67
Love	8,800	528.57	125.68	313.70
Star	5,000	524.14	125.15	310.20

三组查询都扫描电影表，PG 均较快。The 返回记录耗时 209.60 ms，仅计数为 128.13 ms，说明结果传输和处理也是端到端耗时的一部分。Love 和 Star 各约 125 ms；不同关键词还存在匹配计算差异，其耗时差不能全部归因于结果行数。

读多写少负载的吞吐量与尾延迟

8 客户端，10×	PostgreSQL	openGauss
只读吞吐 / 万次 · s^{-1}	9.41	7.35
混合吞吐 / 万次 · s^{-1}	9.43	5.50
只读负载：读 P95 / ms	0.106	0.126
混合负载：读 P95 / ms	0.121	0.218
混合负载：写 P95 / ms	0.180	0.441

各项均取三轮中位数；P95 先在每轮内分别统计读、写请求，再取中位数，未合并三轮样本。

混合负载约 90% 为主键读取，10% 为单行片长更新，每次更新单独同步提交。PG 吞吐基本相当，但读 P95 从 0.106 升至 0.121 ms，写 P95 为 0.180 ms；OG 吞吐下降，读 P95 从 0.126 升至 0.218 ms。总吞吐接近不代表读请求延迟不变。PG 的写入成本是否被客户端瓶颈掩盖，仍需混合负载下的资源干预验证。

04 并发扩展与 CPU 瓶颈分析

在数据库 8 核、客户端 2 核的主键只读负载下，资源干预结果支持 PG 主要受客户端 CPU 限制、OG 主要受数据库 CPU 限制。 本节结合吞吐量、延迟及两端 CPU 占用分析这一判断。

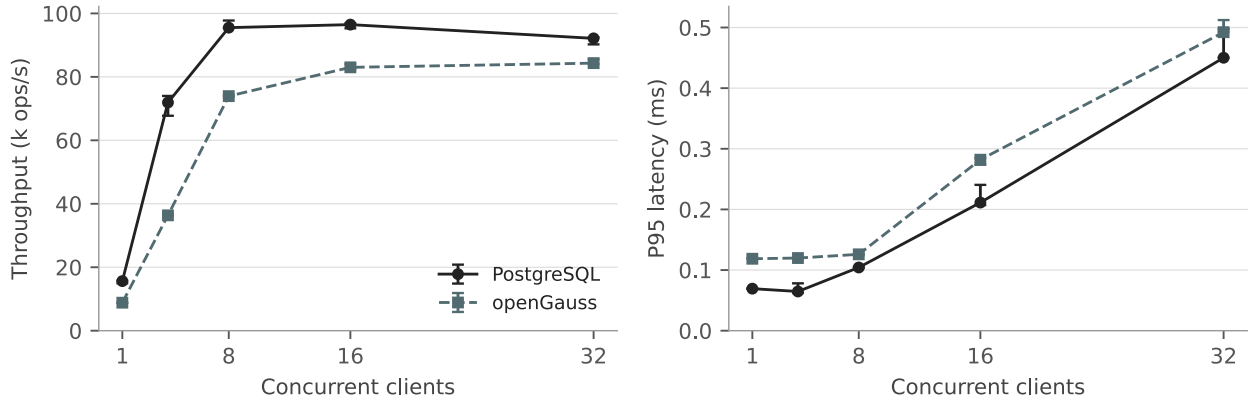


图 2 10 倍数据；数据库 8 核、客户端 2 核。每个客户端收到结果后才发送下一条请求。P95 为约 95% 请求能够完成的时间。

并发从 1 增加到 32 时，PG 吞吐始终较高，但高并发下不再继续增长，P95 延迟也随高并发上升。32 客户端时，PG 客户端用了 2.00 核，数据库仅用 4.06 核；OG 客户端用了 1.88 核，数据库用了 7.99 核。CPU 占用与两端资源限制的判断一致；其影响进一步通过调整 CPU 配额检验。

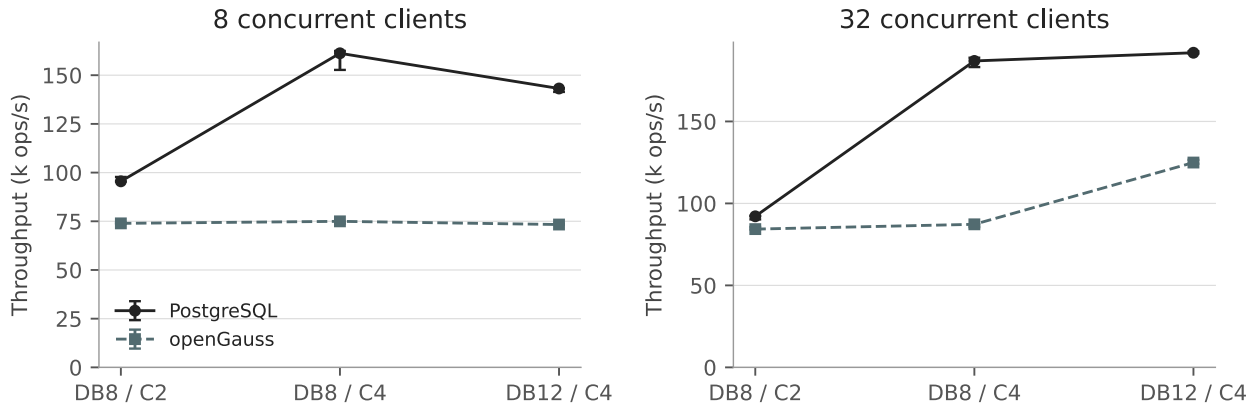


图 3 分别调整客户端与数据库 CPU 配额，其余实验配置相同。DB 为数据库核数，C 为客户端核数。

32 客户端下，保持数据库 8 核，客户端从 2 核增至 4 核，PG 从每秒 9.21 万次增至 18.69 万次；OG 仅从 8.43 增至 8.72 万次。随后保持客户端 4 核，把数据库增至 12 核，OG 升到 12.49 万次，PG 仅小幅升到 19.19 万次。PG 对增加客户端 CPU 的响应更明显，OG 对增加数据库 CPU 的响应更明显。

把关键配置延长到 120 秒，每个条件单独运行一轮，PG 增加客户端核数后的吞吐比为 2.03 倍，OG 增加数据库核数后为 1.42 倍；这次长窗口检查支持原趋势，不提供三轮波动范围。数据库和客户端各分配 8 核的三轮实验中，PG 达到每秒 21.82 万次，数据库接近 8 核满载；OG 为 8.51 万次。因此，原配置下的吞吐平台不能直接视为数据库的处理上限。

05 实验设计与有效性范围

实验控制查询语义、输入数据和更新初始状态，以操作耗时、吞吐量及 P95 延迟评价性能，以 CPU 占用辅助分析资源瓶颈。

数据与资源

课程 `filmdb.sql` 包含国家、电影、人物和演职关系四张表。扩容时复制后三表并平移编号，保持关联关系；国家表始终为 185 行。副本保留原字段分布，用 `replica` 区分重复标题和姓名。

规模	电影	人物	演职关系
1×	9,204	16,489	46,850
10×	92,040	164,890	468,500
100×	920,400	1,648,900	4,685,000

100 倍数据的 TSV 合计约 161.1 MiB，可以全部放入内存。服务器为 Ubuntu 22.04、双路 AMD EPYC 7542、NVMe，同时运行其他任务。基础配置给数据库 8 核、客户端 2 核；两库轮流运行，各限 8 GiB 内存，文件程序、客户端与脚本共用 4 GiB。两库 shared_buffers 为 512 MB、work_mem 为 4 MB，关闭单条查询内部的并行执行。

测量与正确性

串行查询预热一次，并发先预热 5 秒，再测量 20 秒，关键 CPU 干预另用 120 秒复测。通常每组重复三次，报告给中位数；误差线和范围为最小值至最大值。点查先在每轮内求平均，再取三轮中位数。CPU 每 0.5 秒采样，P95 用桶宽约 1% 的直方图统计。

查询耗时包括执行、结果接收和内容摘要计算。标题搜索区分大小写，SQL 使用 `strpos(title,$1)>0`，文件使用 `string::find`。姓名更新每轮前恢复原数据；两库开启 fsync、同步提交和 full_page_writes，文件则先写临时文件、fsync，再替换原文件，使落盘条件接近。

主实验采用历史源码：点查准备时间含扫描 TSV 收集候选编号，执行时间含随机选取编号及字符串转换。当前提交源码先生成完整请求序列，再开始查询计时，并拆分请求准备与后端准备。两版计时边界不同，本文保留历史实测值，不用当前源码的结果替换或反推旧数据。文件索引建立单独测量一次。

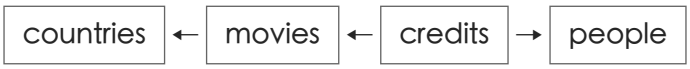
三个实现核对了查询内容、更新结果和表摘要，外键与回滚案例见第 01 节。补充连接覆盖 0、1、22、54 行结果；均匀与热点点查各轮执行 10,000 次、重复 3 轮，热点将 90% 请求集中在前 1% 编号。补充检查时服务器繁忙，结果只作正确性证据，未加入性能排名。外键与回滚通过也不等同于验证了所有隔离级别或崩溃恢复。

实验局限

复制扩容没有引入新的数据分布，热缓存也没有展示大量磁盘读取的代价。短时测试和每轮恢复数据未覆盖长期更新后的旧版本清理。此外，两端内存配额不同，服务器资源非独占，并发采用闭环请求。三轮重复不足以确认微小差异。优先补测更长的共享点查序列，以及同一数据库连接查询有、无人物索引的对照，以检验点查代表性并分离索引收益；超内存数据与持续写入是进一步的扩展方向。

06 实现与复现

四张表通过外键连接；电影和人物通过演职关系形成多对多关系。同一人物可在一部电影中承担不同角色，演职关系主键为 (movieid, peopleid, credited_as)。



箭头从外键所在表指向被引用的表：movies.country → countries.country_code；credits.movieid → movies.movieid；credits.peopleid → people.peopleid。

索引连接与分组聚合

人物索引以 peopleid 为首列，先定位演职记录，再通过电影主键读取详情；文件实现则扫描两张表。参数 $1 为人物编号。

```
CREATE INDEX credits_person_idx ON credits(peopleid, movieid);
SELECT m.movieid, m.title, c.credited_as
FROM movies m JOIN credits c ON m.movieid = c.movieid
WHERE c.peopleid = $1;
```

聚合按国家和年份分组，返回电影数、非空片长之和及其数量；文件端用 std::map 累计同样的结果，两库的诊断执行计划显示 HashAggregate。完整 SQL 内置于提交源码。

姓名更新

筛选名字或姓氏包含大写 T 的人物，替换后提交。first_name 允许为空，筛选用 coalesce，更新保留其空值。文件端使用同样的替换规则，重写人物文件。

```
UPDATE people
SET first_name = replace(first_name, 'T', 'Ttt'),
    surname = replace(surname, 'T', 'Ttt')
WHERE strpos(coalesce(first_name, ''), 'T') > 0
   OR strpos(surname, 'T') > 0;
```

代码与原始记录

主程序使用 C++17、libpq 和 pthread，内置建表 SQL。以下 Ubuntu 命令获取配套数据与脚本，并固定到已验证源码所在的仓库版本；单独编译提交文件时，将源码路径换为所提交的 filmbench.cpp。项目地址：github.com/luanmenglei/dbh_project1。

```
sudo apt-get install g++ libpq-dev python3 git
git clone https://github.com/luanmenglei/dbh_project1.git
cd dbh_project1
git checkout 668d1c787030c21b61bc062c55fca962ef4571d7
c++ -O3 -std=c++17 src/filmbench.cpp \
  -I"$(pg_config --includedir)" -L"$(pg_config --libdir)" \
  -Wl,-rpath,"$(pg_config --libdir)" -lpq -pthread -o filmbench
python3 scripts/prepare.py --scales 1
./filmbench --backend file --op search_rows \
  --data data/generated/s1 --keyword The
```

输出 rows=1563 表示匹配 1,563 部电影。数据库连接参数通过 FILMBENCH_DSN 环境变量传入；以 --backend postgres --op load --data data/generated/s1 导入新建的默认实验 schema，再将上例的后端改为 postgres 查询。openGauss 使用配套 libpq 编译，连接后选择 --backend opengauss。详细部署与补测命令见 README。

主实验源码位于 experiments/baseline-20261012/src/filmbench.cpp；各次实际配置、原始输出及源码快照保存在 results/。当前提交源码已通过三种后端的正确性检查，性能表仍来自历史实验。报告构建脚本从原始记录审查并导出图表。